

EXPERIMENT 29

PROLOG PROGRAM – FORWARD CHAINING

Aim

To implement **Forward Chaining** in Prolog using facts and rules, and derive a conclusion from the given facts.

Algorithm

1. Define the initial facts.
2. Define rules using the rule/2 predicate.
3. Start with the known facts.
4. Check whether the conditions of any rule are satisfied.
5. Add the conclusion of the satisfied rule to the known facts.
6. Repeat the process until no new facts can be added.
7. Check whether the required query is present in the final facts.
8. Display the result.

Prolog Code

```
% Forward Chaining in Prolog
```

```
% Initial facts
```

```
fact(sunny).
```

```
fact(hot).
```

```
% Rules
```

```
rule([sunny, hot], good_weather).
```

```
rule([good_weather], go_outside).
```

```
rule([go_outside], enjoy_day).
```

```
% Forward chaining
```

```
forward_chaining :-
```

```
    findall(X, fact(X), Facts),
```

```
    forward(Facts, FinalFacts),
```

```
    write('Derived Facts: '),
```

```
    write(FinalFacts), nl.
```

```
% Stop when no new fact is produced
```

```
forward(Facts, Facts) :-
```

```
    \+ new_fact(Facts, _), !.
```

```
% Add a new fact
```

```
forward(Facts, FinalFacts) :-
```

```
    new_fact(Facts, NewFact),
```

```
    forward([NewFact|Facts], FinalFacts).
```

```
% Find a new fact using rules
```

```
new_fact(Facts, Conclusion) :-
```

```
    rule(Conditions, Conclusion),
```

```
\+ member(Conclusion, Facts),
```

```
all_true(Conditions, Facts).
```

```
% Check all conditions
```

```
all_true([], _).
```

```
all_true([Condition|Rest], Facts) :-
```

```
    member(Condition, Facts),
```

```
    all_true(Rest, Facts).
```

```
% Query to check whether a fact can be derived
```

```
query(Fact) :-
```

```
    findall(X, fact(X), Facts),
```

```
    forward(Facts, FinalFacts),
```

```
    member(Fact, FinalFacts),
```

```
    write(Fact),
```

```
    write(' can be derived. '), nl.
```

```
query(Fact) :-
```

```
    findall(X, fact(X), Facts),
```

```
    forward(Facts, FinalFacts),
```

```
    \+ member(Fact, FinalFacts),
```

```
    write(Fact),
```

```
    write(' cannot be derived. '), nl.
```

Input

query(rainy).

Output

```
| ?- consult('C:/Users/MOSHIYA/Downloads/exp 29.pl').  
compiling C:/Users/MOSHIYA/Downloads/exp 29.pl for byte code...  
C:/Users/MOSHIYA/Downloads/exp 29.pl compiled, 53 lines read - 4207 bytes written, 0 ms  
  
yes  
| ?- query(rainy).  
rainy cannot be derived.  
  
true ?
```

Result

Thus, the **Forward Chaining algorithm** was successfully implemented in **Prolog**, and the required conclusions were derived from the given facts and rules.